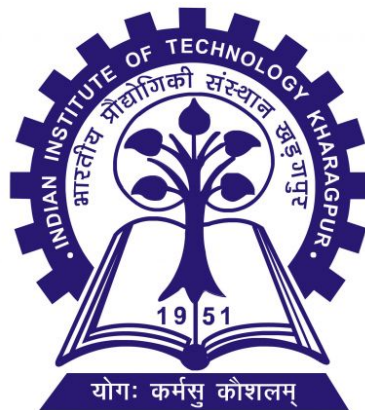


Dept. of Electronics and Electrical Communication Engineering

Indian Institute of Technology, Kharagpur

# VLSI Design Lab



## Experiment 2A

Implementation of a 1-2-4-8-4-2 Pattern Generator using Behavioral and Structural Verilog

Chiradip Biswas 22EC37004

Arup Basak 22EC37021

# 1 Introduction

The objective of this experiment is to design and implement a digital counter that generates the specific repetitive sequence:  $1, 2, 4, 8, 4, 2, \dots$ . The output  $y$  is a 4-bit signal. Two distinct design methodologies are employed to implement this logic using Verilog HDL:

1. **Minimum Flip-Flop Approach:** Utilizes a Finite State Machine (FSM) with the minimum number of state bits required to track the sequence, with output logic derived combinatorially.
2. **Direct Output Approach:** Utilizes the output register bits themselves as the primary state variables, utilizing a "one-hot" style shift mechanism with an auxiliary direction flag.

Both behavioral and structural descriptions are implemented for each method. The designs are synthesized to analyze resource utilization (LUTs, Flip-Flops) and timing constraints (Critical Path Delay).

## 2 Method 1: Minimum Flip-Flop Implementation

### 2.1 Theory and State Encoding

The target sequence contains 6 distinct steps in one full period:  $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 4 \rightarrow 2$ . To implement this using the minimum number of storage elements, we determine the number of necessary states,  $N = 6$ . The minimum number of flip-flops ( $n$ ) required is calculated as:

$$2^{n-1} < N \leq 2^n \implies 2^2 < 6 \leq 2^3 \quad (1)$$

Thus, 3 Flip-Flops are required. We define the state vector as  $Q = \{q_2, q_1, q_0\}$ .

The states are encoded sequentially from 0 to 5 to simplify the transition logic. A combinational decoder is then used to map these states to the required 4-bit output  $y$ .

Table 1: State Transition and Output Table (Min-FF)

| State Name | State Code ( $q_2q_1q_0$ ) | Next State | Output ( $y$ ) | Decimal |
|------------|----------------------------|------------|----------------|---------|
| A          | 000                        | 001        | 0001           | 1       |
| B          | 001                        | 010        | 0010           | 2       |
| C          | 010                        | 011        | 0100           | 4       |
| D          | 011                        | 100        | 1000           | 8       |
| E          | 100                        | 101        | 0100           | 4       |
| F          | 101                        | 000        | 0010           | 2       |

```
1 module cntr1248421(  
2     input clk, output reg [3:0] y  
3 );  
4     localparam [2:0]  
5         a=3'b000,  
6         b=3'b001,  
7         c=3'b010,  
8         d=3'b011,  
9         e=3'b100,  
10        f=3'b101;  
11     reg [2:0] state_reg;
```

```

12
13     always @(posedge clk) begin
14         case(state_reg)
15             a: state_reg<=b;
16             b: state_reg<=c;
17             c: state_reg<=d;
18             d: state_reg<=e;
19             e: state_reg<=f;
20             f: state_reg<=a;
21             default: state_reg<=a;
22         endcase
23     end
24     always @(*) begin
25         case(state_reg)
26             a: y=1;
27             b: y=2;
28             c: y=4;
29             d: y=8;
30             e: y=4;
31             f: y=2;
32         endcase
33     end
34 endmodule

```

## 2.2 Structural Logic Derivation

For the structural implementation, Boolean expressions for the next-state logic ( $D_2, D_1, D_0$ ) and output logic ( $y$ ) were derived (e.g., using K-maps or inspection of the truth table).

### Next State Equations:

- $D_0$ : Toggles on every state transition where the current state is even, effectively  $D_0 = \overline{q_0}$ .
- $D_1$ : High when transitioning from  $000 \rightarrow 001$  or  $100 \rightarrow 101$  requires checking parity and magnitude logic.
- $D_2$ : Controls the transition to states  $E$  and  $F$ .

**Output Equations:** The output  $y[3:0]$  is derived combinationally from the current state  $Q$ :

- $y[0]$  (1): Active only in State A (000).
- $y[1]$  (2): Active in State B (001) and F (101).
- $y[2]$  (4): Active in State C (010) and E (100).
- $y[3]$  (8): Active in State D (011).

```

1 module cntr1248421_st (
2     input  clk,
3     output [3:0] y
4 );

```

```

5
6 wire q2, q1, q0;
7   wire q2n, q1n, q0n;
8   wire d2, d1, d0;
9
10
11  wire d1_xor;
12  wire d2_term1, d2_term2;
13  wire y2_xor;
14
15
16  dff FF0 (.clk(clk), .d(d0), .q(q0), .qn(q0n));
17  dff FF1 (.clk(clk), .d(d1), .q(q1), .qn(q1n));
18  dff FF2 (.clk(clk), .d(d2), .q(q2), .qn(q2n));
19
20
21  assign d0 = q0n;
22
23
24  xor G1_xor (d1_xor, q1, q0);
25  and G1_and (d1, q2n, d1_xor);
26
27
28  and G2_t1 (d2_term1, q2n, q1, q0);
29  and G2_t2 (d2_term2, q2, q1n, q0n);
30  or  G2_or (d2, d2_term1, d2_term2);
31
32
33  and G4 (y[0], q2n, q1n, q0n);
34
35
36  and G5 (y[1], q1n, q0);
37
38
39  xor G6 (y2_xor, q2, q1);
40  and G7 (y[2], q0n, y2_xor);
41
42
43  and G8 (y[3], q2n, q1, q0);
44
45
46 endmodule

```

## 3 Method 2: Direct Output (One-Hot with Direction)

### 3.1 Theory and Shift Logic

In this approach, the output bits  $y[3:0]$  serve directly as the state variables. This resembles a "One-Hot" encoding or a bi-directional shift register. The sequence requires shifting a logic '1' left ( $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ ) and then right ( $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ).

However, the state  $y = 4$  (binary 0100) is ambiguous; it appears twice in the sequence. Looking solely at  $y = 0100$ , the system cannot determine whether the next state should be 8 (shifting left) or 2 (shifting right).

To resolve this ambiguity, an auxiliary Flip-Flop, denoted as  $m$  (Mode/Direction), is introduced:

- $m = 0$ : Up-counting / Shift Left ( $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ )
- $m = 1$ : Down-counting / Shift Right ( $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ )

```
1 module cntr1248421_direct(  
2     input  clk,  
3     output reg [3:0] y  
4 );  
5     reg check;  
6     localparam [3:0]  
7         a=4'b0001,  
8         b=4'b0010,  
9         c=4'b0100,  
10        d=4'b1000;  
11     reg [3:0] state_reg;  
12     always @(posedge clk) begin  
13         case (state_reg)  
14             a: state_reg <= b;  
15             b: state_reg <= check ? a : c;  
16             c: state_reg <= check ? b : d;  
17             d: state_reg <= check ? c : a;  
18             default: state_reg <= a;  
19         endcase  
20     end  
21  
22     always@ (*) begin  
23         if (state_reg[0] == 1) check <= 0;  
24         if (state_reg[3] == 1) check <= 1;  
25         y <= state_reg;  
26     end  
27 endmodule
```

### 3.2 Structural Logic Derivation

The structural logic controls the bi-directional shift and the toggling of the direction bit  $m$ .

#### 1. Direction Logic ( $m$ ):

- Set  $m = 1$  (Down) when the counter reaches 8 ( $y = 1000$ ).
  - Set  $m = 0$  (Up) when the counter reaches 1 ( $y = 0001$ ).
2. **Shift Logic** ( $D_0 \dots D_3$ ): The next state for any bit depends on its neighbors and the current direction  $m$ . For example, bit 2 ( $q_2$ ) receives a '1' if:
- Bit 1 is high AND direction is Up ( $m = 0$ ).
  - Bit 3 is high AND direction is Down ( $m = 1$ ).

This results in a structure where multiplexing logic (implemented via AND-OR gates) selects the input for each Flip-Flop based on the state of  $m$ .

```

1 module cntr1248421_st_direct(
2     input  clk,
3     output [3:0] y
4 );
5
6
7     wire q0, q1, q2, q3;
8     wire q0n, q1n, q2n, q3n;
9     wire m, mn;
10
11
12     wire d0, d1, d2, d3, dm;
13
14
15     wire term_up_2to4, term_up_4to8;
16     wire term_hold_m;
17     wire all_zero;
18
19
20     assign y = {q3, q2, q1, q0};
21
22
23     dff FF0 (.clk(clk), .d(d0), .q(q0), .qn(q0n));
24     dff FF1 (.clk(clk), .d(d1), .q(q1), .qn(q1n));
25     dff FF2 (.clk(clk), .d(d2), .q(q2), .qn(q2n));
26     dff FF3 (.clk(clk), .d(d3), .q(q3), .qn(q3n));
27
28
29     dff FF_M (.clk(clk), .d(dm), .q(m), .qn(mn));
30
31
32     and G0 (all_zero, q0n, q1n, q2n, q3n);
33
34
35     and G1 (term_hold_m, m, q0n);
36     or  G2 (dm, q3, term_hold_m);

```

```

37
38
39     and G3 (term_down_2to1, q1, m);
40     or  G4 (d0, term_down_2to1, all_zero);
41
42
43     and G5 (term_down_4to2, q2, m);
44     or  G6 (d1, q0, term_down_4to2);
45
46
47     and G7 (term_up_2to4, q1, mn);
48     or  G8 (d2, q3, term_up_2to4);
49
50
51     and G9 (d3, q2, mn);
52
53 endmodule

```

## 4 Simulation Results

The same testbench was used for each of four cases:

```

1 module cntr1248421_tb;
2
3     reg clk;
4     wire [3:0] y;
5     //change the instantiation for other three cases
6     cntr1248421_st_direct uut (.clk(clk),.y(y));
7
8     initial begin
9         clk = 0;
10        forever #5 clk = ~clk;
11    end
12
13    initial begin
14        $monitor("Time = %0t ns | Output y (binary) = %b | Output y (
15        decimal) = %d", $time, y, y);
16        #200;
17        $finish;
18    end
19 endmodule

```

For each case output waveform comes as follows:



Figure 1

- **Structural Modeling (Predictable):** In the structural implementation, the next-state logic is derived from Karnaugh maps where unused states are typically treated as "don't cares" ( $X$ ) to minimize gate count. Once the physical logic gates are defined, the path from any spurious state becomes deterministic. If the counter accidentally enters a spurious state (e.g., due to a glitch), the next state is strictly predictable based on the fixed hardware connections.
- **Behavioral Modeling (Unpredictable):** In behavioral modeling, the transitions are often defined using high-level constructs like `case` or `if-else` statements. Unless a `default` case or explicit reset logic is strictly enforced for every possible bit combination, the behavior for spurious states remains undefined in the source code. The synthesis tool is free to optimize these states arbitrarily, rendering the recovery path from a spurious state unpredictable compared to the gate-level design.

## 5 Synthesis Results

The designs were synthesized using Xilinx Vivado targeting the xc7k70tfbg484-3 device. The resource utilization and critical path delay for all four implemented modules are summarized below.

### 5.1 Resource and Timing Data

Table 2 presents the comparative results for the Behavioral and Structural descriptions of both the Minimum Flip-Flop (Min-FF) and Direct Output architectures.

Table 2: Comparison of Resource Utilization and Timing for All Cases

| Design Module            | Method              | Slice LUTs | Slice Regs (FF) | Slice | Delay (ns) |
|--------------------------|---------------------|------------|-----------------|-------|------------|
| Min Flipflop Behavioral  | Min-FF (Behavioral) | 3          | 10              | 2     | 3.574      |
| Min Flipflop Structural  | Min-FF (Structural) | 5          | 3               | 2     | 3.886      |
| Direct Output Behavioral | Direct (Behavioral) | 2          | 5               | 2     | 3.582      |
| Direct Output Structural | Direct (Structural) | 4          | 5               | 1     | 3.518      |

### 5.2 Analysis of Results

The synthesis data in Table 2 reveals distinct trade-offs between the behavioral and structural modeling approaches, particularly regarding register utilization and timing performance.



## Resource Utilization (LUTs and Registers)

- **Register Count Discrepancy:** A significant deviation is observed in the `Min-FF (Behavioral)` case, which consumed 10 Flip-Flops (FFs), compared to only 3 FFs in the `Min-FF (Structural)` implementation. This suggests that the synthesis tool’s optimization strategy for the behavioral description defaulted to a ‘One-Hot’ or similar sparse state encoding scheme to maximize speed, rather than minimizing register count. Additionally, the behavioral model likely inferred separate registers for the output signal  $y$  and the internal state variables.
- **Structural Efficiency:** The `Min-FF (Structural)` design successfully achieved the minimum register usage (3 FFs) as intended, proving that structural modeling provides tighter control over hardware resources when specific constraints (like area or register count) are critical.
- **Logic Utilization (LUTs):** In both architectures, the structural models utilized slightly more Slice LUTs (5 vs. 3 and 4 vs. 2) than their behavioral counterparts. This indicates that the Xilinx Vivado synthesis engine was able to perform superior logic optimization on the abstract behavioral code compared to the fixed gate-level implementation provided in the structural code.

## Timing Performance (Critical Path Delay)

- **Direct vs. Min-FF:** The `Direct Output` architectures generally exhibited lower delays (approx. 3.5 ns) compared to the `Min-FF Structural` case (3.886 ns). This is expected because the `Min-FF` architecture requires additional combinational logic to decode the compressed state (3 bits) into the 4-bit output, adding propagation delay to the critical path.
- **Fastest Design:** The `Direct Output Structural` design yielded the best timing performance with a delay of 3.518 ns. The direct mapping of states to outputs in this architecture eliminates the decoding overhead, optimizing the path from clock to output.

# 6 Discussions

Arup Basak 22EC37021

The implementation and synthesis of the 4-bit counter using two distinct architectures (Minimum Flip-Flop and Direct Output) and two modeling styles (Behavioral and Structural) highlight significant trade-offs in digital system design. The key observations from the synthesis results are as follows:

- **Impact of Abstraction on Resource Usage:** A major discrepancy was observed between the designer’s intent and the synthesis tool’s interpretation in the behavioral models. In the `Min-FF (Behavioral)` case, the design theoretically required only 3 Flip-Flops, yet the tool inferred 10. This suggests the tool defaulted to a sparse encoding (like One-Hot) to maximize frequency, ignoring the area efficiency implicit in the code. Conversely, the structural model forced the tool to use exactly 3 Flip-Flops, proving that gate-level modeling is essential for strict resource control.
- **Area vs. Speed Trade-off:** The “Minimum Flip-Flop” architecture minimizes state storage (3 bits) but incurs a timing penalty due to the combinational decoding logic required to convert the state  $Q$  to the output  $y$ . This explains why the `Min-FF (Structural)` design exhibited the highest critical path delay (3.886 ns). In contrast, the “Direct Output” architecture eliminates the output decoding stage entirely. Consequently, the `Direct Output (Structural)` design achieved the lowest delay (3.518 ns), confirming that reducing logic depth in the critical path improves operating speed.

- **Handling of Spurious States:** The reliability of the counter depends heavily on how the 12 unused (spurious) states are handled. The structural approach yields a deterministic hardware implementation where every possible bit combination leads to a predictable next state based on fixed logic gates. Behavioral modeling, however, leaves the transitions for these states to the synthesis tool’s discretion unless explicitly handled, making the recovery path from a glitch unpredictable compared to the structural design.

## Chiradip Biswas 22EC37004

- **Management of Unused States:** System reliability is highly contingent on the treatment of the 12 undefined (spurious) states. Employing a structural methodology guarantees deterministic hardware, ensuring that any arbitrary state vector resolves to a known next state governed by hardwired logic gates. Conversely, behavioral models delegate these unused transitions to the synthesis engine’s optimization algorithms. Unless explicitly defined in the code, this makes the recovery sequence following a transient glitch highly unpredictable compared to the structural counterpart.
- **Performance versus Area Optimization:** The “Minimum Flip-Flop” approach successfully compresses state storage down to 3 bits, but introduces a latency cost due to the requisite combinational logic that maps the internal state  $Q$  to the final output  $y$ . This additional logic depth accounts for the Min-FF (Structural) implementation experiencing the longest critical path delay of 3.886 ns. On the other hand, the “Direct Output” topology completely bypasses this decoding phase. As a result, the Direct Output (Structural) configuration secured the fastest propagation time at 3.518 ns, demonstrating that minimizing logic layers in the critical path directly accelerates maximum operating speed.
- **Abstraction Level and Hardware Allocation:** Synthesis of the behavioral models revealed a significant divergence between theoretical intent and the tool’s automated inference. While the Min-FF (Behavioral) architecture ostensibly necessitated only 3 Flip-Flops, the compiler instantiated 10. This indicates a compiler preference for sparse state assignments (such as One-Hot encoding) to boost clock speeds, effectively overriding the intended area efficiency. In stark contrast, the structural iteration compelled the synthesizer to allocate exactly 3 Flip-Flops, underscoring that explicit gate-level descriptions are paramount for absolute control over resource utilization.